
Especificación de requisitos de software

Proyecto: Hoodie Shop – E-commerce
Revisión 1.0



Marzo 2026

Ficha del documento

Fecha	Revisión	Autor	Verificado dep. calidad.
03/03/2026	1.0	Nicolas Camilo Bocanegra Vaca	Camilo Bocanegra

Documento validado por las partes en fecha: 03/03/2026

Por el cliente	Por la empresa suministradora
Fdo. D./ Dña Nicolas Camilo Bocanegra Vaca	Fdo. D./Dña Nicolas Camilo Bocanegra Vaca



Contenido

<u>FICHA DEL DOCUMENTO</u>	3
<u>CONTENIDO</u>	4
<u>1 INTRODUCCIÓN</u>	6
<u>1.1 Propósito</u>	6
<u>1.2 Alcance</u>	6
<u>1.3 Personal involucrado</u>	6
<u>1.4 Definiciones, acrónimos y abreviaturas</u>	6
<u>1.5 Referencias</u>	6
<u>1.6 Resumen</u>	6
<u>2 DESCRIPCIÓN GENERAL</u>	7
<u>2.1 Perspectiva del producto</u>	7
<u>2.2 Funcionalidad del producto</u>	7
<u>2.3 Características de los usuarios</u>	7
<u>2.4 Restricciones</u>	7
<u>2.5 Suposiciones y dependencias</u>	7
<u>2.6 Evolución previsible del sistema</u>	7
<u>3 REQUISITOS ESPECÍFICOS</u>	7
<u>3.1 Requisitos comunes de los interfaces</u>	8
<u>3.1.1 Interfaces de usuario</u>	8
<u>3.1.2 Interfaces de hardware</u>	8
<u>3.1.3 Interfaces de software</u>	8
<u>3.1.4 Interfaces de comunicación</u>	8
<u>3.2 Requisitos funcionales</u>	8
<u>3.2.1 Requisito funcional 1</u>	9
<u>3.2.2 Requisito funcional 2</u>	9
<u>3.2.3 Requisito funcional 3</u>	9
<u>3.2.4 Requisito funcional n</u>	9
<u>3.3 Requisitos no funcionales</u>	9
<u>3.3.1 Requisitos de rendimiento</u>	9
<u>3.3.2 Seguridad</u>	9
<u>3.3.3 Fiabilidad</u>	9
<u>3.3.4 Disponibilidad</u>	9



3.3.5	Mantenibilidad	10
3.3.6	Portabilidad	10
3.4	Otros requisitos	10
4	APÉNDICES	10

1 Introducción

Este documento constituye la Especificación de Requisitos de Software (SRS) del sistema Hoodie Shop, una plataforma de comercio electrónico desarrollada con Python/Flask y MongoDB Atlas, orientada a la venta de hoodies con gestión de inventario, pedidos y confirmación por Whatsapp.

1.1 Propósito

El propósito de este documento es definir de manera clara, completa y verificable todos los requisitos funcionales y no funcionales del sistema Hoodie Shop, conforme al estándar IEEE Std830-1998.

Audiencia a la que va dirigida:

- **Desarrolladores:** Para comprender que debe construirse y como debe comportarse el sistema.
- **Testers:** Para diseñar casos de pruebas que verifiquen el cumplimiento de los requisitos.
- **Cliente:** Para validar que el sistema cubre sus necesidades reales.
- **Docentes:** Para verificar el alcance y la completitud del proyecto.

1.2 Alcance

El producto a desarrollar se denomina **Hoodie Shop**.

Hoodie Shop es una aplicación web de comercio electrónico que permite:

- **A clientes:** Explorar catálogo de productos, gestionar carrito de compras, realizar pedidos con pago contra entrega y hacer seguimiento de sus pedidos.
- **A administradores:** Gestionar inventario de productos (CRUD completo), gestionar y actualizar estado de los pedidos, visualizar estadísticas del negocio y exportar reportes en CSV.

El sistema NO incluye en esta versión:

- Pasarelas de pago en línea (tarjetas, PSE, Nequi, etc.)
- Integración con sistemas de logística externos.
- Aplicación móvil nativa.
- Notificaciones automáticas por email.

1.3 Personal involucrado

Nombre	Nicolas Camilo Bocanegra Vaca
Rol	Desarrollador Full Stack / Analista de Requisitos
Categoría profesional	Estudiante Tecnólogo Desarrollo y análisis de Software
Responsabilidades	Análisis, diseño, desarrollo, pruebas y documentación del sistema completo.
Información de contacto	nicolas.bocanegra@pi.edu.co
Aprobación	Autor y responsable del proyecto

1.4 Definiciones, acrónimos y abreviaturas

- **SRS:** Software Requirements Specification – Especificación de Requisitos de Software
- **RF:** Requisito Funcional

- **RNF:** Requisito No Funcional
- **CRUD:** Create, Read, Update, Delete – Operaciones básicas sobre datos
- **COP:** Pesos Colombianos – Moneda utilizada del sistema
- **MongoDB:** Base de datos NoSQL orientada a documentos
- **Atlas:** Servicio cloud de MongoDB para base de datos administradas
- **Flask:** Microframework web desarrollado en Python
- **Blueprint:** Modulo de rutas de Flask que permite organizar las URLs por funcionalidad
- **Bootstrap:** Framework CSS para diseño de interfaces responsivas
- **Jinja2:** Motor de plantillas HTML integrado en Flask
- **API:** Application Programming Control – Interfaz de programación de aplicaciones
- **RBAC:** Role-Based Access Control – Control de acceso basado en roles
- **Hash:** Resultado de aplicar una función criptográfica unidireccional a una contraseña
- **PBKDF2:** Password-Based Key Derivation Function 2 – Algoritmo de hasing seguro
- **Carrito:** Estructura temporal en sesión que almacena los productos seleccionados antes del pedido
- **Checkout:** Proceso de confirmación y finalización de una compra
- **Admin:** Usuario con rol `admin` con privilegios de administración del sistema
- **Cliente:** Usuario con rol `user` que utiliza la tienda para realizar compras
- **Placeholder:** Imagen genérica mostrada cuando un producto no tiene foto asignada
- **Whatsapp:** Aplicación de mensajería usada para confirmar pedidos manualmente
- **CVS:** Comma Separated Values – Archivo de texto plano para exportar datos tabulares
- **Soft Delete:** Eliminación lógica: el registro se marca como inactivo, pero no se borra físicamente
- **Context Processor:** Función de Flask que inyecta variables globales disponibles en todos los templates
- **Factory Pattern:** Patrón de diseño para crear instancias de la aplicación con configuración variable
- **g:** Objeto global de Flask que almacena datos durante el ciclo de vida de un request
- **Venv:** Entorno virtual de Python para aislamiento de dependencias
- **dotenv:** Librería para cargar variables de entorno desde un archivo `.env`

1.5 Referencias

Referencia	Título	Ruta	Fecha	Autor
IEEE 830	IEEE Recommended for Software Requirements Specifications	https://ieeexplore.ieee.org/document/720574	1998	IEEE
Flask	Flask Documentation v3.0	https://flask.palletsprojects.com/	2024	Pallets Projects
Flask-Login	Flask-Login Documentation v0.6.3	https://flask-login.readthedocs.io/	2024	Flask-Login
MongoDB	MongoDB Atlas Documentation	https://docs.mongodb.com/	2024	MongoDB Inc.
PyMongo	PyMongo 4.6.1 Documentation	https://pymongo.readthedocs.io/	2024	MongoDB Inc.
Bootstrap	Bootstrap 5.3 Documentation	https://getbootstrap.com/docs/5.3	2024	Bootstrap Team
Python	Python 3.14	https://docs.python.org/3/	2024	Python

	Documentation			Software Foundation
Werkzeug	Werkzeug 3.0.1 Documentation	https://werkzeug.palletsprojects.com/	2024	Pallets Projects
GitHub	Repositorio del proyecto	https://github.com/DjKiller07FT/hoodie-shop	2026	Nicolas Camilo Bocanegra Vaca
Whatsapp	Whatsapp Click To Chat API	https://faq.whatsapp.com/425247423114725	2024	Meta

1.6 Resumen

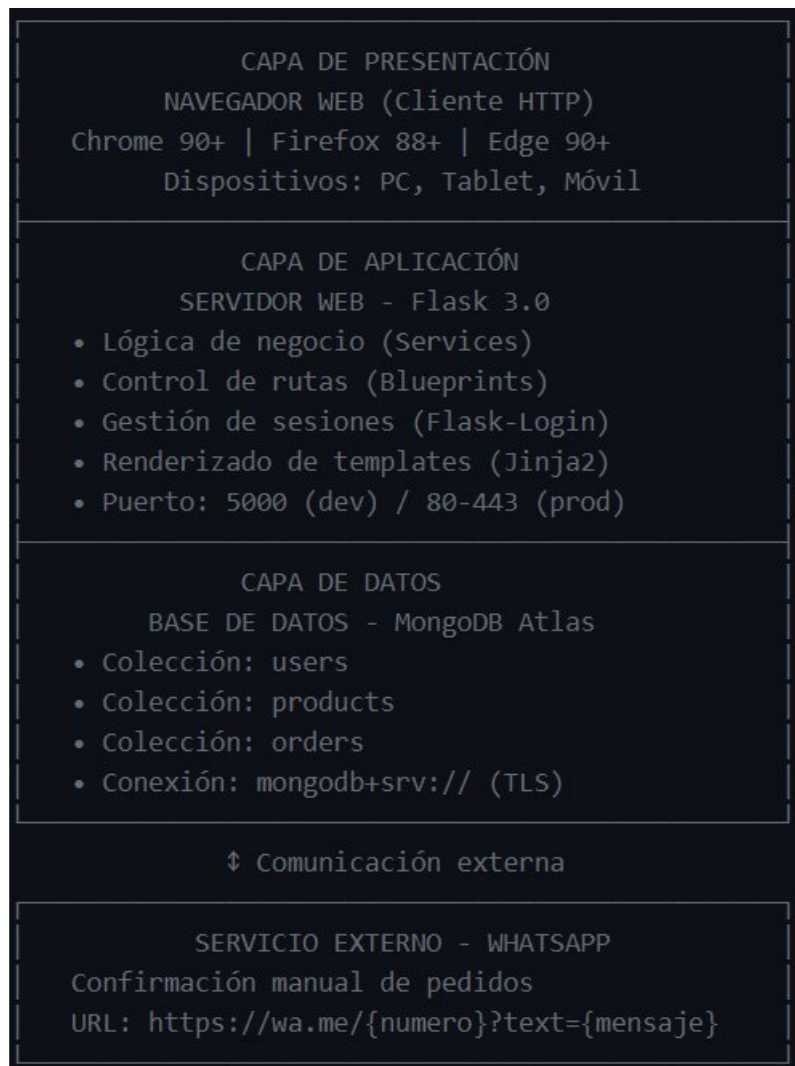
El presente documento esta organizado en **4 secciones principales**:

- **Sección 1 – Introducción:** Proporciona el propósito, alcance, personal involucrado, definiciones, referencias y organización del documento.
- **Sección 2 – Descripción general:** Describe el contexto del producto, su perspectiva dentro del ecosistema tecnológico, funcionalidades principales, tipos de usuario, restricciones, suposiciones y evolución futura previstas.
- **Sección 3 – Requisitos específicos:** Contiene la descripción detallada y completa de **20 requisitos funcionales** y **20 requisitos no funcionales**, todos justificados con código real del repositorio. Incluye interfaces de usuario, hardware, software y comunicación.
- **Sección 4 – Apéndices:** Incluye el modelo de datos MongoDB, la estructura del proyecto, las tecnologías y versiones utilizadas, los casos de uso principales y la matriz de trazabilidad de requisitos.

2 Descripción general

2.1 Perspectiva del producto

Hoodie Shop es un producto **independiente** que no forma parte de un sistema mayor preexistente. Opera como una aplicación web de tres capas:



Punto de entrada: “run.py” – Instancia la aplicación con “create_app(config_name)” y ejecuta el servidor Flask en “host=’0.0.0.0’”, puerto configurable mediante variable de entorno “PORT” (por defecto “5000”)

2.2 Funcionalidad del producto

Módulo de Autenticación (“/auth/*”):

- Registro de nuevos usuarios con Login automático
- Inicio de sesión con redirección por rol
- Cierre de sesión seguro

Módulo de Catalogo / Tienda (“/catalog”, “/producto/*”, “/cart/*”):

- Catalogo de productos activos con búsqueda por texto
- Vista detallada de producto con selector de talla, color y cantidad
- Carrito de compras en sesión del servidor

Módulo de Pedidos (“/checkout”, “/user/*”):

- Proceso de checkout con validación de stock
- Generación de número de pedido único
- Confirmación de Whatsapp con mensaje prellenado
- Historial de pedidos del cliente con seguimiento de estado

Módulo Administrativo (“ /admin/* ”):

- Dashboard con estadísticas y alertas de stock bajo
- CRUD completo de productos con gestión de imágenes
- Gestión y cambio de estado de pedidos con historial
- Exportación de pedidos a CVS

2.3 Características de los usuarios

Usuario tipo: Cliente

Tipo de usuario	Cliente / Comprador
Formación	Educación básica o media, sin conocimientos técnicos
Habilidades	Uso básico de navegador web y Whatsapp
Actividades	Explorar catálogo, agregar al carrito, realizar pedidos, consulta estado de pedidos, editar perfil

Usuario tipo: Administrador

Tipo de usuario	Administrador / Gestor de tienda
Formación	Educación media o superior, conocimientos básicos en gestión de tiendas online
Habilidades	Manejo de navegadores web, gestión de inventarios y pedidos
Actividades	Crear/editar/desactivar productos, gestionar y actualizar pedidos, revisar Dashboard, exportar reportes

2.4 Restricciones

1. **Método de pago único:** Solo se soporta pago contra entrega. No se integran pasarelas de pago en línea.
2. **Confirmación manual:** Los pedidos requieren confirmación manual del cliente vía Whatsapp. No hay notificaciones automáticas.
3. **Lenguaje backend:** Exclusivamente Python 3.10+ con Flask 3.0.
4. **Base de datos:** Exclusivamente MongoDB (Atlas cloud o local).
5. **Dependencias de internet:** El sistema requiere conexión a internet activa(servidor Flask + MongoDB Atlas).
6. **Navegadores soportados:** Chrome 90+, Firefox 88+, Edge 90+, Safari 14+. Sin soporte para internet explorer.
7. **Imágenes:** Tamaño máximo 5MB. Formatos jpg, jpeg, png, gif, webp.
8. **Sistema operativo servidor:** Desarrollo en Windows 10/11. Producción recomendada en Ubuntu 20.04+.

2.5 Suposiciones y dependencias

Suposiciones:

- Python 3.10+ disponible en el servidor de despliegue.
- Conexión a internet estable con un mínimo 10 Mbps en el servidor.
- Los usuarios finales tienen acceso a un dispositivo con navegador web moderno.
- El administrador tiene acceso a Whatsapp para recibir confirmaciones de pedidos.
- MongoDB Atlas mantiene disponibilidad en el tier M0 (gratuito).

Dependencias:

- MongoDB Atlas a Sin conexión, el sistema no puede operar
- “wa.me” (Whatsapp) a El enlace de confirmación dejaría de funcionar
- Bootstrap CDN a La interfaz perdería estilos si se usa CDN externo sin internet
- Python 3.10+ a Cambios de versión incompatible requiere ajustes en el código

- Librerías en “ requirements.txt ” à Actualizaciones de API pueden requerir cambios en el código

2.6 Evolución previsible del sistema

Versión 1.1 – Corto plazo:

- Sistema de cupones y descuentos
- Notificaciones automáticas por email (confirmación de pedido)
- Galería de múltiples imágenes por producto
- Alertas automáticas de stock bajo por email

Versión 1.2 – Mediano plazo:

- Sistema de reviews y calificaciones de productos
- Lista de deseos
- Filtros avanzados en catalogo (precio, talla, color)
- Múltiples direcciones de envío por usuario

Versión 2.0 – Largo plazo:

- Integración con pasarela de pago (Stripe, PayU, Mercado Pago)
- Aplicación móvil nativa (Android, iOS)
- Integración con logística (tracking automático)
- Panel de analíticas avanzadas con graficas
- Inventario con alertas automáticas y reposición sugerida

3 Requisitos específicos

3.1 Requisitos comunes de los interfaces

3.1.1 Interfaces de usuario

- **Diseño Responsivo:** Bootstrap 5.3 adaptable a móvil (320px+), Tablet (768px+) y escritorio (1200px+).
- **Esquema de colores:** Negro “ #000000 ” como primario, blanco “ #FFFFFF ” como fondo, amarillo/dorado “ #FFC107 ” como acento de botones.
- **Tipografía:** Fuente sans-serif Bootstrap, tamaño base 16px.
- **Navegación:** Barra fija en la parte superior con logo, enlaces y acceso a carrito/usuario.
- **Mensajes feedback:** Alerta flash Bootstrap (“ success ” verde, “ danger ” rojo, “ warning ” amarillo, “ info ” azul) con animación “ fadeIn ” definida en “ custom.css ”.
- **Formularios:** Validación en cliente (“ main.js ”) y en servidor (servidor y rutas).
- **Accesibilidad:** Atributos “ alt ” en imágenes y “ label ” en formularios.

3.1.2 Interfaces de hardware

- **Servidor de desarrollo:** Mínimo 4GB, 2 núcleos, 10GB en disco. Sistema actual: Windows10/11, Python 3.14.
- **Servidor de producción:** VPS con mínimo 1GB RAM, 1 vCPU, 25GB SSD (compatible con Render, Railway y PythonAnywhere).
- **Dispositivo del usuario:** Cualquier dispositivo con navegador web moderno.
- **Almacenamiento de imágenes:** Sistema de archivos local en “ app/static/uploads ”.

3.1.3 Interfaces de software

Software	Versión	Propósito
Python	3.14	Leguaje backend

Flask	3.0.0	Framework web
Werkzeug	3.0.1	Hashing, seguridad, utilidades HTTP
Flask-Login	0.6.3	Gestión de sesiones de usuarios
PyMongo	4.6.1	Driver de conexión a MongoDB
dnspython	2.4.2	Resolución DNS para conexión Atlas (“mongodb+srv//”)
pythondotenv	1.0.0	Cargan de variables de entorno desde “.env”
email-validator	2.1.0	Validación de formato de emails
python-dateutil	2.8.2	Manejo avanzado de fechas
pytest	7.4.3	Framework de pruebas unitarias
pytest-flask	1.3.0	Extensión de pytest para Flask
flask-cors	4.0.0	Cross-Origin Resource Sharing
Bootstrap	5.3.x	Framework CSS frontend
Jinja2	3.x	Motor de templates (incluido en Flask)
MongoDB Atlas	Cloud MO	Base de datos NoSQL en la nube

3.1.4 Interfaces de comunicación

- **HTTP/HTTPS:** Toda comunicación entre navegador y servidor Flask. Puerto “5000” en desarrollo, “80/443” en producción.
- **MongoDB Wire Protocol:** Comunicación con MongoDB Atlas mediante URI “mongodb+srv://” sobre TCP/IP con TLS.
- **Whatsapp API (wa.me):** URL de redirección unidireccional “https://wa.me/{numero}?text={mensaje_encoded}”. El mensaje se codifica con “urllib.parse.quote()” en “app/services/whatsapp_services.py”.
- **Sesiones HTTP:** Cookies firmadas con “SECRET_KEY” gestionadas por Flask-Login.

3.2 Requisitos funcionales

3.2.1 Requisito funcional 1

Número de requisito	RF – 01	
Nombre de requisito	Registro de nuevo usuario cliente	
Tipo	☒ Requisito	☐ Restricción
Fuente del requisito	Cliente / Usuario Final	
Prioridad del requisito	☒ Alta/Esencial	☐ Media/Deseado ☐ Baja/ Opcional

Descripción:

Ruta “GET/POST /auth/register” en “app/routes/auth.py”, protegida con “@logout_required” para evitar acceso de usuarios ya autenticados.

Entradas:

- Nombre completo (obligatorio)
- Email (obligatorio, formato valido, único en sistema)
- Teléfono (obligatorio, formato colombiano)
- Dirección (obligatorio)
- Ciudad (obligatorio)
- Contraseña (obligatorio, mínimo 6 características)
- Confirmar contraseña (debe coincidir)

Proceso:

1. Valida coincidencia de contraseñas en la ruta.

2. "auth_services.register_user()" " valida formato de email, unicidad y longitud de contraseña
3. Aplica hash " PBKDF2-SHA256 " con " set_password()" " en " app/models/user.py "
4. Crea el usuario con " rol = 'user' " en colección " user " de MongoDB.
5. Inicia sesión automáticamente con " login_user(user, remember=True) ".
6. Redirige al catalogo

Salidas:

- **Éxito:** Sesión activa, mensaje flash de bienvenida, redirección a " /catalog ".
- **Error:** Mensaje descriptivo indicando campo y tipo de error

3.2.2 Requisito funcional 2

Número de requisito	RF – 02		
Nombre de requisito	Inicio de sesión de usuario		
Tipo	☒ Requisito	☐ Restricción	
Fuente del requisito	Cliente / Administrador		
Prioridad del requisito	☒ Alta/Esencial	☐ Media/Deseado	☐ Baja/ Opcional

Descripción:

Ruta " GET/POST /auth/login " protegida con " @logout_required ". Implementado en " app/routes/auth.py "

Entradas:

Email, contraseña, checkbox "Recordarme" (opcional)

Proceso:

1. "auth_service.login_user(email, password)" busca usuario en MongoDB.
2. Verifica contraseña con "check_password_hash()".
3. "login_user(user, remember=remember)" crea la sesión.
4. Redirección por rol:
 - "rol = 'admin'" → "/admin/dashboard"
 - "rol = 'user'" → "/catalog"
 - Si hay parámetro "?next=" → redirige a la URL solicitada.

Salidas:

Éxito: Sesión activa, mensaje "¡Bienvenido {nombre}!".

Error: Mensaje genérico "Email o contraseña incorrectos".

3.2.3 Requisito funcional 3

Número de requisito	RF – 03		
Nombre de requisito	Cierre de sesión seguro		
Tipo	☒ Requisito	☐ Restricción	
Fuente del requisito	Cliente / Administrador		
Prioridad del requisito	☒ Alta/Esencial	☐ Media/Deseado	☐ Baja/ Opcional

Descripción:

Ruta "GET /auth/logout" protegida con " @login_required ". Destruye la sesión activa del usuario.

Proceso:

1. "logout_user()" invalida la sesión de Flask-Login.
2. Redirige al catálogo.

Salidas:

Éxito: Mensaje "Sesión cerrada correctamente", redirección a `/catalog`.

3.2.4 Requisito funcional 4

Número de requisito	RF – 04		
Nombre de requisito	Catalogo de productos con búsqueda por texto		
Tipo	☒ Requisito	☐ Restricción	
Fuente del requisito	Cliente		
Prioridad del requisito	☒ Alta/Esencial	☐ Media/Deseado	☐ Baja/ Opcional

Descripción:

Ruta pública `GET /catalog` (accesible sin autenticación). Implementado en `app/routes/shop.py` → `catalog()`.

Entradas:

Parámetro URL `?q=texto` (opcional).

Proceso:

- Con `?q=`: `product\_service.buscar\_productos(query)` — búsqueda regex insensible a mayúsculas en `nombre` y `descripcion`.
- Sin `?q=`: `product\_service.get\_all\_products(solo\_activos=True)` — todos los productos con `activo = True`.

Salidas:

- Grid responsivo con imagen (o placeholder), nombre y precio COP.
- Sin resultados: Mensaje informativo.

3.2.5 Requisito funcional 5

Número de requisito	RF – 05		
Nombre de requisito	Vista detallada de producto		
Tipo	☒ Requisito	☐ Restricción	
Fuente del requisito	Cliente		
Prioridad del requisito	☒ Alta/Esencial	☐ Media/Deseado	☐ Baja/ Opcional

Descripción:

Ruta pública `GET /product/<product\_id>`. Implementado en `app/routes/shop.py` → `product\_detail()`.

Proceso:

1. Busca producto por `ObjectId` en MongoDB.
2. Verifica que exista y `activo = True`.
3. Calcula tallas disponibles (solo las que tienen `stock > 0`).

Salidas:

- Imagen, nombre, descripción, precio COP.
- Selectores de talla (solo con stock disponible), color y cantidad.
- Botón "Agregar al Carrito".
- Enlace de consulta por WhatsApp.
- Producto inactivo/inexistente: Redirección al catálogo con mensaje.

3.2.6 Requisito funcional 6

Número de requisito	RF – 06		
---------------------	---------	--	--

Nombre de requisito	Agregar producto al carrito de compras		
Tipo	☒ Requisito	☐ Restricción	
Fuente del requisito	Cliente		
Prioridad del requisito	☒ Alta/Esencial	☐ Media/Deseado	☐ Baja/ Opcional

Descripción:

Ruta `POST /cart/add`. Disponible para usuarios no autenticados. Implementado en `app/routes/shop.py` → `add\_to\_cart()`.

Entradas:

`product\_id`, `talla`, `color`, `cantidad`.

Proceso:

1. Obtiene el producto de MongoDB.
2. Verifica stock con `product.tiene\_stock(talla, cantidad)`.
3. Inicializa `session['cart'] = []` si no existe.
4. Crea el item: `{product\_id, nombre, talla, color, cantidad, precio\_unitario, subtotal, imagen}`.
5. Si ya existe el mismo `product\_id + talla + color`: suma la cantidad.
6. Marca `session.modified = True`.

Salidas:

Éxito: Flash "Producto agregado al carrito", redirección al carrito.

Error Sin Stock: Flash "Stock insuficiente para talla {talla}".

Error: Producto no encontrado: Redirección al catálogo.

3.2.7 Requisito funcional 7

Número de requisito	RF – 07		
Nombre de requisito	Visualización y gestión del carrito de compras		
Tipo	☒ Requisito	☐ Restricción	
Fuente del requisito	Cliente		
Prioridad del requisito	☒ Alta/Esencial	☐ Media/Deseado	☐ Baja/ Opcional

Descripción:

Rutas `GET /cart` y `GET /cart/remove/<index>`. Implementado en `app/routes/shop.py`.

Funcionalidades:

- Lista todos los items con imagen, nombre, talla, color, cantidad, precio unitario y subtotal.
- Total general calculado: `sum(item['subtotal'] for item in cart\_items)`.
- Eliminar items por índice con `remove\_from\_cart(index)`.
- Actualizar cantidades con `update\_cart()`.
- Botón "Vaciar Carrito" y "Proceder al Pago"
- Contador dinámico en navbar inyectado globalmente por `context\_processor`.

Salidas:

- Tabla de items con total en COP.
- Carrito vacío: Mensaje y botón para ir al catálogo.

3.2.8 Requisito funcional 8

Número de requisito	RF – 08		
Nombre de requisito	Proceso de checkout y creación de pedido		

Tipo	☒ Requisito	☐ Restricción
Fuente del requisito	Cliente	
Prioridad del requisito	☒ Alta/Esencial	☐ Media/Deseado ☐ Baja/ Opcional

Descripción:

Ruta `GET/POST /checkout` protegida con `@login\_required`. Implementado en `app/routes/shop.py` → `checkout()`.

Entradas (formulario):

`nombre`, `telefono`, `direccion`, `ciudad` (pre-cargados con datos del usuario), `notas` (opcional).

Proceso:

1. Verifica que el carrito no esté vacío.
2. Valida stock de cada item nuevamente con `product.tiene\_stock()`.
3. `order\_service.create\_order(user\_id, items, direccion\_envio)` crea el pedido en MongoDB.
4. `product\_service.reducir\_stock(product\_id, talla, cantidad)` descuenta el stock.
5. Vacía el carrito con `session.pop('cart', None)`.
6. Genera enlace WhatsApp con `WhatsAppService.generar\_enlace\_whatsapp(order, user)`.
7. Renderiza `order\_success.html` con número de pedido y botón WhatsApp

Validaciones en `order\_service.create\_order()`:

- Items no vacíos.
- Campos requeridos: `nombre`, `telefono`, `direccion`, `ciudad`.

Salidas:

Éxito: Página con resumen de pedido, número único y botón WhatsApp verde.

Error Stock insuficiente: Flash error + redirección al carrito.

Error Sin sesión: Redirección a login.

3.2.9 Requisito funcional 9

Número de requisito	RF – 09	
Nombre de requisito	Integración con Whatsapp para confirmación de pedido	
Tipo	☒ Requisito	☐ Restricción
Fuente del requisito	Cliente / Administrador	
Prioridad del requisito	☒ Alta/Esencial	☐ Media/Deseado ☐ Baja/ Opcional

Descripción:

Servicio en `app/services/whatsapp\_service.py` → clase `WhatsAppService`.

Métodos:

- `generar\_mensaje\_pedido(order, user)`: Genera mensaje con emoji con número de pedido, datos del cliente, lista de ítems (`nombre`, `talla`, `color`, `cantidad`, `subtotal`) y total COP.
- `generar\_enlace\_whatsapp(order, user)`: Codifica el mensaje con `urllib.parse.quote()` y retorna URL `https://wa.me/{numero}?text={mensaje\_encoded}`.
- `generar\_mensaje\_consulta(producto, user)`: Genera enlace de consulta sobre un producto.

Configuración:

- Número en `.env` → `WHATSAPP\_NUMBER` con prefijo `57` (Colombia).

- Botón en `order\_success.html` abre en nueva pestaña (`target="\_blank"`)

3.2.10 Requisito funcional 10

Número de requisito	RF – 10		
Nombre de requisito	Historial y detalle de pedidos del cliente		
Tipo	☒ Requisito	☐ Restricción	
Fuente del requisito	Cliente		
Prioridad del requisito	☒ Alta/Esencial	☐ Media/Deseado	☐ Baja/ Opcional

Descripción:

Rutas `GET /user/orders` y `GET /user/orders/<order\_id>` en `app/routes/user.py`, protegidas con `@login\_required`.

Proceso:

1. `order\_service.get\_orders\_by\_user(user\_id)`: Pedidos del usuario ordenados por fecha descendente.
2. `order\_detail()`: Verifica que `str(order.user\_id) == current\_user.get\_id()` antes de mostrar el detalle.

Salidas:

Éxito Lista de pedidos con: número, fecha, total COP, badge de estado

Detalle: ítems comprados, datos de envío, historial completo de estados con fechas.

Error Pedido ajeno: Flash "No tienes permiso para ver este pedido" + redirección.

3.2.11 Requisito funcional 11

Número de requisito	RF – 11		
Nombre de requisito	Edición del perfil personal del usuario		
Tipo	☒ Requisito	☐ Restricción	
Fuente del requisito	Cliente		
Prioridad del requisito	☒ Alta/Esencial	☐ Media/Deseado	☐ Baja/ Opcional

Descripción:

Ruta `GET/POST /user/profile` en `app/routes/user.py`, protegida con `@login\_required`.

Entradas:

- Nombre, teléfono, dirección, ciudad.
- Cambio de contraseña (opcional): requiere contraseña actual + nueva (mínimo 6 caracteres).

Proceso:

- Actualiza campos con `auth\_service.update\_user(user\_id, \*\*updates)`.
- Si se provee nueva contraseña: verifica la actual con `current\_user.check\_password()`. Solo actualiza si es correcta.

Salidas:

Éxito: Flash "Perfil actualizado exitosamente", redirección al perfil.

Contraseña incorrecta: Flash "Contraseña actual incorrecta".

3.2.12 Requisito funcional 12

Número de requisito	RF – 12		
---------------------	---------	--	--

Nombre de requisito	Panel de control con estadísticas del negocio		
Tipo	☒ Requisito	☐ Restricción	
Fuente del requisito	Administrador		
Prioridad del requisito	☒ Alta/Esencial	☐ Media/Deseado	☐ Baja/ Opcional

Descripción:

Ruta ``GET /admin/dashboard`` protegida con ``@login_required`` + ``@admin_required``. Implementado en ``app/routes/admin.py`` → ``dashboard()``.

Métricas mostradas:

- Total de pedidos (todos los estados).
- Ventas totales acumuladas en COP (aggregation pipeline de MongoDB).
- Conteo de pedidos por cada estado válido.
- Productos con ``stock_total < 10`` unidades (alerta de bajo stock).
- Últimos 10 pedidos recientes.

3.2.13 Requisito funcional 13

Número de requisito	RF – 13		
Nombre de requisito	Gestión completa de productos		
Tipo	☒ Requisito	☐ Restricción	
Fuente del requisito	Administrador		
Prioridad del requisito	☒ Alta/Esencial	☐ Media/Deseado	☐ Baja/ Opcional

Descripción:

Rutas en ``app/routes/admin.py``, todas protegidas con ``@login_required`` + ``@admin_required``.

Ruta	Método	Función
<code>`GET /admin/products`</code>	GET	Lista todos con filtros <code>`?q=`</code> y <code>`?inactive=true`</code>
<code>`GET/POST /admin/products/new`</code>	GET/POST	GET/POST Crear nuevo producto
<code>`GET/POST /admin/products/<id>/edit`</code>	GET/POST	Editar producto existente
<code>`GET /admin/products/<id>/delete`</code>	GET	Desactivar producto (soft delete)
<code>`GET /admin/products/<id>/restore`</code>	GET	Restaurar producto desactivado

Servicios:

``ProductService`` → ``create_product()``, ``update_product()``, ``delete_product()`` (soft), ``restore_product()``.

Imágenes:

``save_image()`` de ``helpers.py`` con ``secure_filename()`` y nombres únicos con contador.

3.2.14 Requisito funcional 14

Número de requisito	RF – 14		
Nombre de requisito	Gestión y cambio de estado de pedidos		
Tipo	☒ Requisito	☐ Restricción	
Fuente del requisito	Administrador		
Prioridad del requisito	☒ Alta/Esencial	☐ Media/Deseado	☐ Baja/ Opcional

Descripción:

Rutas en `app/routes/admin.py`, todas protegidas con `@login\_required` + `@admin\_required`.

Ruta	Método	Función
`GET /admin/orders`	GET	Lista todos con filtro `?estado=`
`GET /admin/orders/<id>`	GET	Detalle del pedido con info del cliente
`POST /admin/orders/<id>/change-status`	POST	Cambiar estado del pedido

Estados válidos (`Order.ESTADOS\_VALIDOS`):

Estado	Significado
`RECIBIDO`	Pedido recibido, pendiente confirmación
`ALISTAMIENTO`	Confirmado, preparando productos
`ENVIO`	Enviado, en camino al cliente
`ENTREGADO`	Entregado exitosamente

Proceso de cambio:

`order\_service.cambiar\_estado(order\_id, nuevo\_estado, admin\_id)` registra el cambio en `historial\_estados` con fecha y `cambiado\_por`.

3.2.15 Requisito funcional 15

Número de requisito	RF – 15	
Nombre de requisito	Exportar pedidos en formato CSV	
Tipo	☒ Requisito	☐ Restricción
Fuente del requisito	Administrador	
Prioridad del requisito	☐ Alta/Esencial	☒ Media/Deseado ☐ Baja/ Opcional

Descripción:

Ruta `GET /admin/orders/export` con filtro `?estado=`. Implementado en `app/routes/admin.py` → `orders\_export()`.

Proceso:

1. `order\_service.exportar\_pedidos\_csv(estado\_filtro)` retorna datos formateados
2. `csv.DictWriter` escribe en buffer `StringIO`.
3. Respuesta con `Content-Disposition: attachment; filename=pedidos.csv`.

Salidas:

Archivo descargable `pedidos.csv` con columnas: Número Pedido, Fecha, Cliente, Teléfono, Ciudad, Total, Estado.

3.2.16 Requisito funcional 16

Número de requisito	RF – 16	
Nombre de requisito	Control automático de inventario por talla	
Tipo	☒ Requisito	☐ Restricción
Fuente del requisito	Sistema	
Prioridad del requisito	☒ Alta/Esencial	☐ Media/Deseado ☐ Baja/ Opcional

Descripción:

Implementado en `app/services/product\_service.py` y `app/models/product.py`.

Funciones:

- `reducir\_stock(product\_id, talla, cantidad)`: Descuenta stock al confirmar pedido. Valida disponibilidad antes de descontar.
- `aumentar\_stock(product\_id, talla, cantidad)`: Incrementa stock (uso administrativo).
- `product.tiene\_stock(talla, cantidad)`: Verificación previa al agregar al carrito y al checkout.
- `product.get\_stock\_total()`: Suma stock de todas las tallas para alertas de bajo stock (umbral < 10).

3.2.17 Requisito funcional 17

Número de requisito	RF – 17		
Nombre de requisito	Búsqueda y filtrado de productos		
Tipo	☒ Requisito	☐ Restricción	
Fuente del requisito	Cliente / Administrador		
Prioridad del requisito	☐ Alta/Esencial	☒ Media/Deseado	☐ Baja/ Opcional

Descripción:

Implementado en `app/services/product\_service.py`.

- `buscar\_productos(query)`: Regex insensible a mayúsculas sobre `nombre` y `descripcion` con `activo = True`.
- `get\_all\_products(solo\_activos)`: Listado completo o solo activos.
- `filtrar\_productos(\*\*filtros)`: Combinación de múltiples filtros dinámicos.
- Panel admin: `?inactive=true` para ver productos desactivados, `?q=texto` para búsqueda administrativa.

3.2.18 Requisito funcional 18

Número de requisito	RF – 18		
Nombre de requisito	Generación de numero de pedido único garantizado		
Tipo	☒ Requisito	☐ Restricción	
Fuente del requisito	Sistema		
Prioridad del requisito	☒ Alta/Esencial	☐ Media/Deseado	☐ Baja/ Opcional

Descripción:

Implementado en `app/models/order.py` → `generar\_numero\_pedido()` y `app/services/order\_service.py` → `\_generar\_numero\_pedido\_unico()`.

Proceso:

1. Formato: `ORD-{YYYY}-{NNNNNN}` (año actual + 6 dígitos aleatorios).
2. `\_generar\_numero\_pedido\_unico()` verifica en MongoDB que el número no exista.
3. Reintenta si hay colisión (bucle `while True`).
4. El campo tiene índice `unique=True` en MongoDB (definido en `setup\_indexes.py`).

3.2.19 Requisito funcional 19

Número de requisito	RF – 19		
Nombre de requisito	Verificación de propiedad en acceso a pedidos		
Tipo	☒ Requisito	☐ Restricción	
Fuente del requisito	Sistema / Seguridad		

Prioridad del requisito	☒ Alta/Esencial	☐ Media/Deseado	☐ Baja/ Opcional
-------------------------	---	--	---

Descripción:

Implementado en `app/routes/user.py` → `order\_detail()`. Previene que un usuario vea pedidos ajenos.

Proceso:

```
```python
if str(order.user_id) != current_user.get_id():
 flash('No tienes permiso para ver este pedido', 'danger')
 return redirect(url_for('user.orders'))
```
```

Salidas:

Acceso no autorizado: Flash de error + redirección.

Autorizado: Muestra detalle completo del pedido.

3.2.20 Requisito funcional 20

| | | | |
|-------------------------|--|---|---|
| Número de requisito | RF – 20 | | |
| Nombre de requisito | Generación de estadísticas del negocio para el Dashboard | | |
| Tipo | ☒ Requisito | ☐ Restricción | |
| Fuente del requisito | Administrador | | |
| Prioridad del requisito | ☐ Alta/Esencial | ☒ Media/Deseado | ☐ Baja/ Opcional |

Descripción:

Implementado en `app/services/order\_service.py` → `get\_estadisticas()`.

Métricas:

- `total\_pedidos`: `count\_documents({})`.
- `pedidos\_por\_estado`: `count\_documents({'estado': estado})` por cada estado.
- `total\_ventas`: Aggregation pipeline `\$group → \$sum \$total`.
- Últimos 10 pedidos: `get\_all\_orders()[10]`.
- Productos bajo stock: Filtrado en ruta por `stock\_total < 10`.

3.3 Requisitos no funcionales

3.3.1 Requisitos de rendimiento

| | | | |
|-------------------------|---|--|---|
| Número de requisito | RNF – 01 | | |
| Nombre de requisito | Tiempo de respuesta del sistema | | |
| Tipo | ☒ Requisito | ☐ Restricción | |
| Fuente del requisito | Especificación técnica | | |
| Prioridad del requisito | ☒ Alta/Esencial | ☐ Media/Deseado | ☐ Baja/ Opcional |

- El **95%** de las páginas deben cargar en menos de **2 segundos**.
- Búsquedas de productos en menos de **1 segundo**.
- Índices MongoDB definidos en `setup\_indexes.py`: `email` (único), `rol`, `nombre`, `activo`, `precio`, `numero\_pedido` (único), `user\_id`, `estado`, `created\_at` (desc).
- Imágenes limitadas a **5MB** (`MAX\_CONTENT\_LENGTH = 5 \* 1024 \* 1024` en `config.py`).
- Sistema soporta mínimo **10 usuarios simultáneos** sin degradación perceptible.

| | | | |
|---------------------|----------|--|--|
| Número de requisito | RNF – 02 | | |
|---------------------|----------|--|--|

| | | |
|-------------------------|--|---|
| Nombre de requisito | Escalabilidad mediante paginación configurable | |
| Tipo | ☒ Requisito | ☐ Restricción |
| Fuente del requisito | Especificación técnica | |
| Prioridad del requisito | ☐ Alta/Eencial | ☒ Media/Deseado ☐ Baja/ Opcional |

- ``PRODUCTS_PER_PAGE = 12`` y ``ORDERS_PER_PAGE = 20`` en ``app/config.py``.
- Valores configurables mediante variables de entorno sin modificar código.
- MongoDB Atlas permite escalado horizontal en tiers superiores.

3.3.2 Seguridad

| | | |
|-------------------------|--|--|
| Número de requisito | RNF – 03 | |
| Nombre de requisito | Control de acceso basado en roles | |
| Tipo | ☒ Requisito | ☐ Restricción |
| Fuente del requisito | Especificación técnica | |
| Prioridad del requisito | ☒ Alta/Eencial | ☐ Media/Deseado ☐ Baja/ Opcional |

- ``@admin_required`` en ``app/utils/decorators.py``: Verifica ``current_user.is_admin()`` en cada request a ``/admin/*``.
- ``@logout_required``: Bloquea acceso a login/registro si ya hay sesión activa.
- Las verificaciones ocurren en **cada request**, no solo al iniciar sesión.
- Redirección automática con mensaje de error para accesos no autorizados.

| | | |
|-------------------------|--|--|
| Número de requisito | RNF – 04 | |
| Nombre de requisito | Hashing seguro de contraseñas | |
| Tipo | ☒ Requisito | ☐ Restricción |
| Fuente del requisito | Especificación técnica | |
| Prioridad del requisito | ☒ Alta/Eencial | ☐ Media/Deseado ☐ Baja/ Opcional |

- Contraseñas almacenadas exclusivamente con hash ``PBKDF2-SHA256`` (Werkzeug).
- ``set_password()`` y ``check_password()`` en ``app/models/user.py``.
- ``SECRET_KEY`` desde variable de entorno, nunca hardcodeda.
- Las contraseñas no aparecen en respuestas HTTP ni en logs.

| | | |
|-------------------------|--|--|
| Número de requisito | RNF – 05 | |
| Nombre de requisito | Gestión segura de sesiones de usuario | |
| Tipo | ☒ Requisito | ☐ Restricción |
| Fuente del requisito | Especificación técnica | |
| Prioridad del requisito | ☒ Alta/Eencial | ☐ Media/Deseado ☐ Baja/ Opcional |

- ``SESSION_TYPE = 'filesystem'``, ``SESSION_PERMANENT = True``, ``PERMANENT_SESSION_LIFETIME = timedelta(days=7)`` en ``app/config.py``.
- Sesiones firmadas criptográficamente con ``SECRET_KEY``.
- ``login_manager.login_view = 'auth.login'`` redirige a login si la sesión expira.

| | | |
|-------------------------|--|--|
| Número de requisito | RNF – 06 | |
| Nombre de requisito | Validación y seguridad en carga de imágenes | |
| Tipo | ☒ Requisito | ☐ Restricción |
| Fuente del requisito | Especificación técnica | |
| Prioridad del requisito | ☒ Alta/Eencial | ☐ Media/Deseado ☐ Baja/ Opcional |

- ``allowed_file()``: Valida extensión contra ``{'png', 'jpg', 'jpeg', 'gif', 'webp'}``.

- ``save_image()``: Usa ``secure_filename()`` para sanitizar nombres y prevenir path traversal.
- Genera nombres únicos con contador incremental si ya existe el archivo.
- Almacenamiento en ``app/static/uploads/'`, aislado del código ejecutable.

3.3.3 Fiabilidad

| | | |
|-------------------------|---|---|
| Número de requisito | RNF – 07 | |
| Nombre de requisito | Manejo robusto de errores del sistema | |
| Tipo | ☒ Requisito | ☐ Restricción |
| Fuente del requisito | Especificación técnica | |
| Prioridad del requisito | ☐ Alta/Esencial | ☒ Media/Deseado ☐ Baja/ Opcional |

- Errores de conexión MongoDB capturados con ``ConnectionFailure`` en ``test_mongo_connection()``.
- Todas las operaciones críticas de BD usan ``try/except``.
- Errores registrados con ``app.logger.error()`` sin exponer detalles al usuario.
- MTBF estimado: **720 horas** (30 días) de operación continua sin intervención.

| | | |
|-------------------------|---|--|
| Número de requisito | RNF – 08 | |
| Nombre de requisito | Trazabilidad completa del historial de pedidos | |
| Tipo | ☒ Requisito | ☐ Restricción |
| Fuente del requisito | Administrador / Cliente | |
| Prioridad del requisito | ☒ Alta/Esencial | ☐ Media/Deseado ☐ Baja/ Opcional |

- Cada cambio de estado registra en ``historial_estados``: estado, fecha/hora, ``cambiado_por`` (ObjectId del admin).
- Historial inmutable: solo se agregan entradas, nunca se eliminan.
- Los pedidos nunca se eliminan físicamente de MongoDB.
- Consultable por cliente (``/user/orders/<id>``) y por admin (``/admin/orders/<id>``).

3.3.4 Disponibilidad

| | | |
|-------------------------|---|---|
| Número de requisito | RNF – 09 | |
| Nombre de requisito | Disponibilidad del sistema en producción | |
| Tipo | ☒ Requisito | ☐ Restricción |
| Fuente del requisito | Especificación técnica | |
| Prioridad del requisito | ☐ Alta/Esencial | ☒ Media/Deseado ☐ Baja/ Opcional |

- Desarrollo: **100%** durante horas de trabajo activo.
- Producción: Mínimo **99%** anual (~7.2 horas de inactividad permitidas).
- MongoDB Atlas: Sujeto a disponibilidad garantizada por el servicio.
- Verificación de conexión al iniciar con ``test_mongo_connection()``.
- Producción: Uso de ``gunicorn`` con múltiples workers para tolerancia a fallos.

| | | |
|-------------------------|---|--|
| Número de requisito | RNF – 10 | |
| Nombre de requisito | Separación de entornos mediante configuración | |
| Tipo | ☒ Requisito | ☐ Restricción |
| Fuente del requisito | Especificación técnica | |
| Prioridad del requisito | ☒ Alta/Esencial | ☐ Media/Deseado ☐ Baja/ Opcional |

- ``DevelopmentConfig``: ``DEBUG = True``, ``TESTING = False``.
- ``ProductionConfig``: ``DEBUG = False``, ``TESTING = False``.
- Selección mediante ``FLASK_ENV`` sin modificar código fuente.

- ``create_app(config_name)`` en ``run.py``: ``config_name = os.getenv('FLASK_ENV', 'development')``.

3.3.5 Mantenibilidad

| | | |
|-------------------------|---|--|
| Número de requisito | RNF – 11 | |
| Nombre de requisito | Mantenibilidad mediante arquitectura modular | |
| Tipo | ☒ Requisito | ☐ Restricción |
| Fuente del requisito | Especificación técnica | |
| Prioridad del requisito | ☒ Alta/Esencial | ☐ Media/Deseado ☐ Baja/ Opcional |

- 5 capas bien definidas: ``models/``, ``services/``, ``routes/``, ``utils/``, ``templates/``.
- 4 Blueprints independientes: ``auth``, ``shop``, ``admin``, ``user``.
- 4 Servicios desacoplados: ``AuthService``, ``ProductService``, ``OrderService``, ``WhatsAppService``.
- Factory Pattern (``create_app()``) para instanciación configurable.
- Docstrings en todas las funciones con descripción, ``Args`` y ``Returns``.
- Dependencias en ``requirements.txt``, revisión trimestral recomendada.

| | | |
|-------------------------|---|---|
| Número de requisito | RNF – 12 | |
| Nombre de requisito | Capacidad de pruebas unitarias | |
| Tipo | ☒ Requisito | ☐ Restricción |
| Fuente del requisito | Especificación técnica | |
| Prioridad del requisito | ☐ Alta/Esencial | ☒ Media/Deseado ☐ Baja/ Opcional |

- Carpeta ``tests/`` con 3 módulos: ``test_auth.py``, ``test_products.py``, ``test_orders.py``.
- Factory Pattern permite crear instancia de app en modo test con BD aislada.
- Servicios inyectados con la conexión DB, facilitando el uso de mocks.
- Ejecución: ``python -m pytest tests/``

| | | |
|-------------------------|---|--|
| Número de requisito | RNF – 13 | |
| Nombre de requisito | Gestión de código fuente con Git y GitHub | |
| Tipo | ☒ Requisito | ☐ Restricción |
| Fuente del requisito | Especificación técnica | |
| Prioridad del requisito | ☒ Alta/Esencial | ☐ Media/Deseado ☐ Baja/ Opcional |

- Repositorio: ``https://github.com/DjKiller07FT/hoodie-shop``.
- ``gitignore`` excluye ``venv/``, ``env``, ``__pycache__/``, ``app/static/uploads/*``.
- ``env.example`` como plantilla documentada para nuevos despliegues.
- Convención de commits semánticos: ``feat:``, ``fix:``, ``docs:``, ``style:``, ``refactor:``, ``test:``, ``chore:``.
- Composición: Python 49.5%, HTML 46.4%, JavaScript 2.8%, CSS 1.3%.

3.3.6 Portabilidad

| | | |
|-------------------------|---|---|
| Número de requisito | RNF – 14 | |
| Nombre de requisito | Portabilidad entre plataformas | |
| Tipo | ☒ Requisito | ☐ Restricción |
| Fuente del requisito | Especificación técnica | |
| Prioridad del requisito | ☐ Alta/Esencial | ☒ Media/Deseado ☐ Baja/ Opcional |

- Compatible con Windows 10+, Ubuntu 20.04+ y macOS 12+ sin modificaciones.
- Python es multiplataforma por diseño.
- MongoDB Atlas elimina dependencia de motor de BD local.

- `requirements.txt` garantiza replicabilidad exacta del entorno.
- ~5% del código puede requerir ajustes por rutas de archivos en diferentes SO.

| | |
|-------------------------|--|
| Número de requisito | RNF – 15 |
| Nombre de requisito | Configurabilidad total mediante variables de entorno |
| Tipo | ☒ Requisito ☐ Restricción |
| Fuente del requisito | Especificación técnica |
| Prioridad del requisito | ☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/ Opcional |

- `python-dotenv` carga el archivo `.env` en `app/config.py` y `seed\_admin.py`.
- Variables configurables: `SECRET\_KEY`, `MONGO\_URI`, `UPLOAD\_FOLDER`, `MAX\_CONTENT\_LENGTH`, `WHATSAPP\_NUMBER`, `PRODUCTS\_PER\_PAGE`, `ORDERS\_PER\_PAGE`, `ADMIN\_EMAIL`, `ADMIN\_PASSWORD`, `ADMIN\_NOMBRE`, `PORT`, `FLASK\_DEBUG`.
- Ninguna credencial sensible está hardcodeada en el código fuente.
- `.env.example` documenta todas las variables para nuevos despliegues.

| | |
|-------------------------|--|
| Número de requisito | RNF – 16 |
| Nombre de requisito | Preparado para despliegue en plataformas cloud |
| Tipo | ☒ Requisito ☐ Restricción |
| Fuente del requisito | Especificación técnica |
| Prioridad del requisito | ☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional |

- Despliegue posible sin modificaciones en: Render, Railway, PythonAnywhere, Heroku.
- `run.py` configura host `0.0.0.0` y puerto desde variable `PORT` para compatibilidad con plataformas cloud.
- `os.makedirs(app.config['UPLOAD\_FOLDER'], exist\_ok=True)` crea carpeta `uploads/` automáticamente.
- `README.md` incluye instrucciones completas de instalación y despliegue.

| | |
|-------------------------|--|
| Número de requisito | RNF – 17 |
| Nombre de requisito | Usabilidad e interfaz web responsiva |
| Tipo | ☒ Requisito ☐ Restricción |
| Fuente del requisito | Cliente / Usuario final |
| Prioridad del requisito | ☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/ Opcional |

- Bootstrap 5.3: Responsivo en móvil (320px+), tablet (768px+), escritorio (1200px+).
- `app/static/js/main.js`: Validación de contraseñas en tiempo real, formateo de precios, preview de imagen antes de subir.
- `app/static/css/custom.css`: Animación `fadeIn` para alertas flash, hover en tablas.
- Navbar fija con contador dinámico de items del carrito (inyectado por `context\_processor`).

| | |
|-------------------------|--|
| Número de requisito | RNF – 18 |
| Nombre de requisito | Localización para Colombia |
| Tipo | ☒ Requisito ☐ Restricción |
| Fuente del requisito | Cliente |
| Prioridad del requisito | ☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional |

- Idioma: Español colombiano en toda la interfaz, mensajes y validaciones

- Moneda: COP con `formato\_moneda\_cop()` en `app/utils/helpers.py`.
- Teléfonos: `validar\_telefono()` para formato colombiano (10 dígitos).
- WhatsApp: Número con prefijo `57` (Colombia), mensajes en español colombiano.
- Sin i18n: Diseñado exclusivamente para operación en Colombia.

| | | |
|-------------------------|--|--|
| Número de requisito | RNF – 19 | |
| Nombre de requisito | Variables globales inyectadas en todos los templates | |
| Tipo | ☒ Requisito | ☐ Restricción |
| Fuente del requisito | Especificación técnica | |
| Prioridad del requisito | ☐ Alta/Esencial | ☒ Media/Deseado ☐ Baja/Opcional |

- `@app.context\_processor` → `inject\_globals()` en `app/\_\_init\_\_.py`.
- Inyecta `cart\_count` (conteo del carrito) disponible en todos los templates automáticamente.
- La navbar en `base.html` muestra el contador actualizado sin código adicional en cada ruta.
- Centraliza datos comunes para evitar duplicación en cada función de vista.

| | | |
|-------------------------|---|---|
| Número de requisito | RNF – 20 | |
| Nombre de requisito | Gestión eficiente de conexión a base de datos | |
| Tipo | ☒ Requisito | ☐ Restricción |
| Fuente del requisito | Especificación técnica | |
| Prioridad del requisito | ☒ Alta/Esencial | ☐ Media/Deseado ☐ Baja/Opcional |

- `get\_db()`: Crea la conexión MongoDB y la almacena en el objeto `g` de Flask (una conexión por request).
- `close\_db(e=None)`: Registrada con `app.teardown\_appcontext(close\_db)`, cierra la conexión al finalizar cada request.
- Previene agotamiento del pool de conexiones en producción.
- Evita conexiones zombie que acumulan memoria en MongoDB Atlas.

3.4 Otros requisitos

Requisitos culturales y regionales

- El sistema usa **español colombiano** como único idioma. Sin soporte de i18n en esta versión.
- Los precios se expresan en **pesos colombianos (COP)**: formato `\$X.XXX.XXX COP`.
- Los números de teléfono siguen el formato colombiano (10 dígitos, comenzando con `3` para celulares).
- Los mensajes de WhatsApp están redactados en español colombiano con emojis de contexto.
- El formato de número de pedido incluye el año actual: `ORD-2026-NNNNNN`.

Requisitos legales

- El sistema **no almacena** datos de tarjetas de crédito ni información bancaria.
- Los datos personales (nombre, email, teléfono, dirección) se almacenan conforme a principios básicos de privacidad.
- Se recomienda en una versión futura agregar política de privacidad y términos y condiciones según la **Ley 1581 de 2012** (Protección de Datos Personales en Colombia) y el **Decreto 1074 de 2015**.
- Las imágenes subidas por el administrador son responsabilidad del propietario del sistema.

4 Apéndices

4.1 Apéndice A: Modelo de datos MongoDB

Colección: `users`

```
``json
{
  "_id": "ObjectId",
  "nombre": "String (requerido)",
  "email": "String (requerido, único, índice)",
  "telefono": "String (requerido)",
  "direccion": "String (requerido)",
  "ciudad": "String (requerido)",
  "password_hash": "String (PBKDF2-SHA256)",
  "rol": "String ('user' | 'admin')",
  "created_at": "DateTime",
  "updated_at": "DateTime"
},
...
```

Colección: `products`

```
``json
{
  "_id": "ObjectId",
  "nombre": "String (requerido, índice)",
  "descripcion": "String (requerido)",
  "precio": "Number (> 0, índice)",
  "stock": {
    "S": "Number (>= 0)",
    "M": "Number (>= 0)",
    "L": "Number (>= 0)",
    "XL": "Number (>= 0)"
  },
  "colores": ["String"],
  "imagen": "String (ruta relativa /static/uploads/ o URL)",
  "activo": "Boolean (índice)",
  "created_at": "DateTime",
  "updated_at": "DateTime"
},
...
```

Colección: `orders`

```
``json
{
  "_id": "ObjectId",
  "numero_pedido": "String (único, índice, formato ORD-YYYY-NNNNNN)",
  "user_id": "ObjectId (ref users, índice)",
  "items": [
    {
      "product_id": "ObjectId",
      "nombre": "String",
      "talla": "String (S|M|L|XL)",
      "color": "String",
      "cantidad": "Number",
      "precio_unitario": "Number",
    }
  ]
},
...
```

```

    "subtotal": "Number",
    "imagen": "String"
  },
],
"total": "Number",
"direccion_envio": {
  "nombre": "String",
  "telefono": "String",
  "direccion": "String",
  "ciudad": "String",
  "notas": "String (opcional)"
},
"estado": "String (RECIBIDO|ALISTAMIENTO|ENVIO|ENTREGADO, índice)",
"historial_estados": [
  {
    "estado": "String",
    "fecha": "DateTime",
    "cambiado_por": "ObjectId"
  }
],
"created_at": "DateTime (índice desc)",
"updated_at": "DateTime"
}

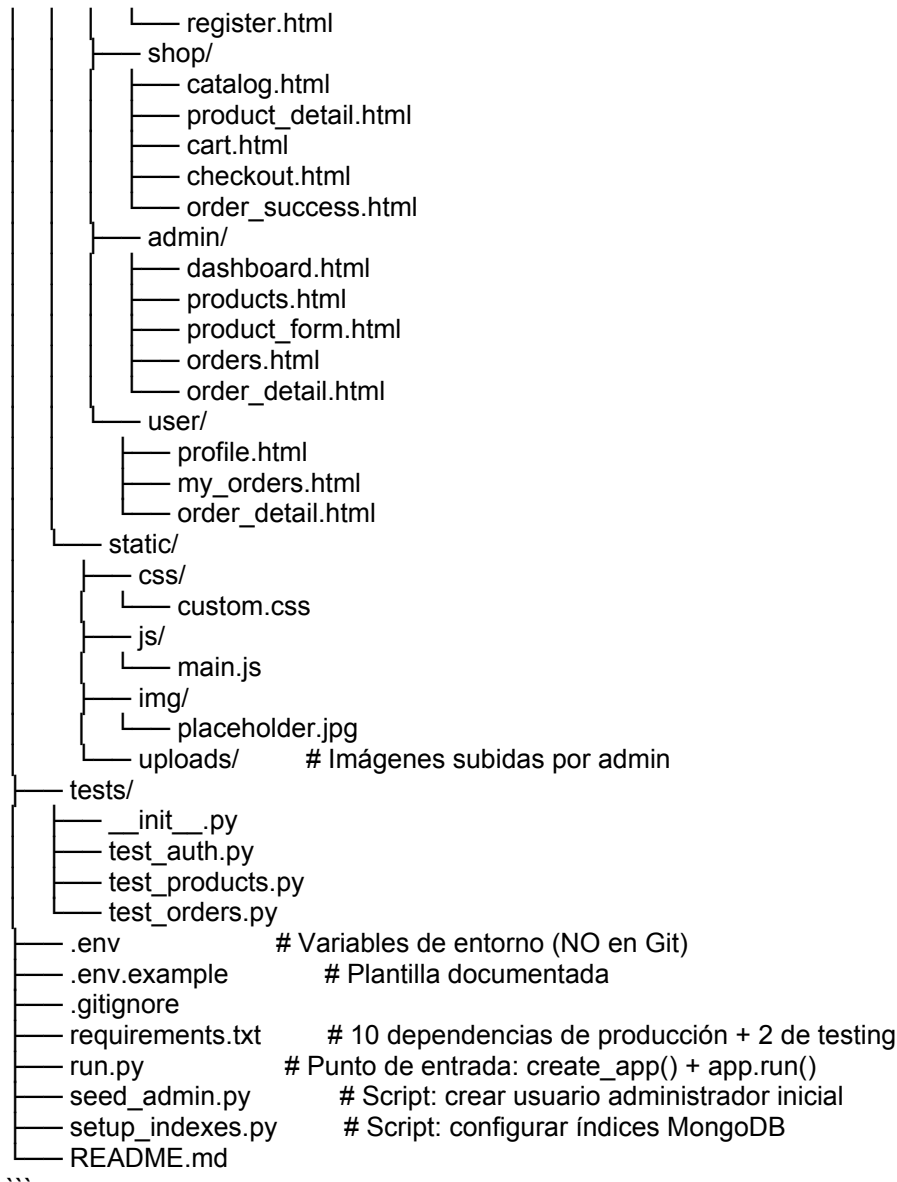
```

4.2 Apéndice B: Estructura del proyecto

```

...
hoodie-shop/
├── app/
│   ├── __init__.py    # Factory Pattern: create_app(), get_db(), close_db()
│   ├── config.py      # DevelopmentConfig, ProductionConfig
│   ├── models/
│   │   ├── __init__.py
│   │   ├── user.py    # Modelo User + Flask-Login
│   │   ├── product.py # Modelo Product + lógica de stock
│   │   └── order.py   # Modelo Order + estados + historial
│   ├── routes/
│   │   ├── __init__.py
│   │   ├── auth.py    # Blueprint auth: /auth/*
│   │   ├── shop.py    # Blueprint shop: /catalog, /product, /cart, /checkout
│   │   ├── admin.py   # Blueprint admin: /admin/*
│   │   └── user.py    # Blueprint user: /user/*
│   ├── services/
│   │   ├── __init__.py
│   │   ├── auth_service.py # AuthService: registro, login, update
│   │   ├── product_service.py # ProductService: CRUD, stock, búsqueda
│   │   ├── order_service.py # OrderService: crear, cambiar estado, CSV
│   │   └── whatsapp_service.py # WhatsAppService: mensajes y enlaces
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── decorators.py # @admin_required, @logout_required
│   │   └── helpers.py    # formato_moneda_cop, allowed_file, save_image
│   └── templates/
│       ├── base.html    # Template base con navbar y flash messages
│       ├── auth/
│       └── login.html

```



4.3 Apéndice C: Tecnologías y versiones (requirements.txt)

```

...
# Framework web
Flask==3.0.0
Werkzeug==3.0.1

# Autenticación
Flask-Login==0.6.3

# Base de datos
pymongo==4.6.1
dnspython==2.4.2

# Variables de entorno
python-dotenv==1.0.0
  
```

Formularios y validación
email-validator==2.1.0

Utilidades
python-dateutil==2.8.2

Testing
pytest==7.4.3
pytest-flask==1.3.0

Desarrollo
flask-cors==4.0.0
...

4.4 Apéndice D: Casos de uso principales

4.4.1 CU-01: Comprar un producto (Cliente)

...

Actor: Cliente autenticado
Precondición: Sesión activa, productos disponibles

Flujo principal:

1. Cliente navega al catálogo (/catalog)
2. Cliente busca un producto (opcional, ?q=texto)
3. Cliente hace clic en "Ver Detalle"
4. Cliente selecciona talla, color y cantidad
5. Cliente hace clic en "Agregar al Carrito"
6. Sistema verifica stock disponible
7. Cliente accede al carrito (/cart)
8. Cliente hace clic en "Proceder al Pago"
9. Sistema verifica sesión activa
10. Cliente revisa/edita datos de envío
11. Cliente confirma el pedido
12. Sistema crea el pedido, reduce stock, vacía carrito
13. Sistema muestra página de éxito con botón WhatsApp
14. Cliente envía confirmación por WhatsApp

Flujo alternativo 6a (sin stock):

Sistema muestra "Stock insuficiente para talla X"
Cliente selecciona otra talla

Flujo alternativo 9a (sin sesión):

Sistema redirige al login con ?next=/checkout
Cliente inicia sesión → regresa al checkout

...

4.4.2 CU-02: Gestionar estado de pedido (Administrador)

...

Actor: Administrador autenticado
Precondición: Sesión activa con rol=admin

Flujo principal:

1. Admin accede a /admin/dashboard
2. Admin navega a /admin/orders
3. Admin filtra pedidos por estado (opcional)
4. Admin hace clic en un pedido

5. Admin selecciona nuevo estado del formulario
6. Admin confirma el cambio
7. Sistema actualiza estado y registra historial
8. Sistema muestra flash de confirmación
- ...

4.4.3 CU-03: Crear producto (Administrador)

...

Actor: Administrador autenticado

Precondición: Sesión activa con rol=admin

Flujo principal:

1. Admin accede a /admin/products/new
2. Admin completa el formulario (nombre, descripción, precio, stock por talla, colores)
3. Admin sube imagen (opcional)
4. Admin hace clic en "Crear Producto"
5. Sistema valida los datos
6. Sistema guarda imagen con nombre único seguro
7. Sistema crea el producto en MongoDB con activo=True
8. Sistema redirige a la lista de productos con mensaje de éxito
- ...

4.5 Apéndice E: Matriz de trazabilidad de requisitos

| Requisito | Archivo fuente | Función / Clase | Ruta HTTP |
|-----------|--|---|-----------|
| RF-01 | `routes/auth.py` | `register()` | |
| RF-02 | `routes/auth.py` | `login()` | |
| RF-03 | `routes/auth.py` | `logout()` | |
| RF-04 | `routes/shop.py` | `catalog()` | |
| RF-05 | `routes/shop.py` | `product_detail()` | |
| RF-06 | `routes/shop.py` | `add_to_cart()` | |
| RF-07 | `routes/shop.py` | `cart()`, `remove_from_cart()` | |
| RF-08 | `routes/shop.py` | `checkout()` | |
| RF-09 | `services/whatsapp_service.py` | `WhatsAppService` | |
| RF-10 | `routes/user.py` | `orders()`, `order_detail()` | |
| RF-11 | `routes/user.py` | `profile()` | |
| RF-12 | `routes/admin.py` | `dashboard()` | |
| RF-13 | `routes/admin.py` | `product_new()` | |
| RF-14 | `routes/admin.py` | `order_change_status()` | |
| RF-15 | `routes/admin.py` | `orders_export()` | |
| RF-16 | `services/product_service.py` | `reducir_stock()`, `aumentar_stock()` | |
| RF-17 | `services/product_service.py` | `buscar_productos()`, `filtrar_productos()` | |
| RF-18 | `models/order.py` | `generar_numero_pedido()` | |
| RF-19 | `routes/user.py` | `order_detail()` | |
| RF-20 | `services/order_service.py` | `get_estadisticas()` | |
| RNF-01 | `setup_indexes.py` | Índices MongoDB | |
| RNF-02 | `PRODUCTS_PER_PAGE`, `ORDERS_PER_PAGE` | `ORDERS_PER_PAGE` | |
| RNF-03 | `utils/decorators.py` | `admin_required`, `logout_required` | |
| RNF-04 | `models/user.py` | `set_password()`, `check_password()` | |

| | | | |
|--------|------------------------------|---|--|
| RNF-05 | `config.py` | `SESSION_*`,
`PERMANENT_SESSION_LIFETIME` | |
| RNF-06 | `utils/helpers.py` | `allowed_file()`, `save_image()` | |
| RNF-07 | `__init__.py` | `test_mongo_connection()`,
`try/except` | |
| RNF-08 | `models/order.py` | `historial_estados` | |
| RNF-09 | `__init__.py` | `test_mongo_connection()` | |
| RNF-10 | `config.py` | `DevelopmentConfig`,
`ProductionConfig` | |
| RNF-11 | Toda la arquitectura | Blueprints + Services | |
| RNF-12 | `tests/` | `test_auth.py`, `test_products.py`,
`test_orders.py` | |
| RNF-13 | `.gitignore`, `.env.example` | Control de versiones Git | |
| RNF-14 | `requirements.txt` | Python multiplataforma | |
| RNF-15 | `config.py` | variables de entorno | |
| RNF-16 | `run.py` | `host='0.0.0.0'`, `PORT` env var | |
| RNF-17 | `static/js/main.js` | Validaciones cliente + Bootstrap 5 | |
| RNF-18 | `utils/helpers.py` | `formato_moneda_cop()`,
`validar_telefono()` | |
| RNF-19 | `__init__.py` | `inject_globals()` context_processor | |
| RNF-20 | `__init__.py` | `get_db()`, `close_db()`,
`teardown_appcontext` | |